Check for updates

ACM DIGITAL LIBRARY

Association for Computing Machinery

acm open

Latest updates: https://dl.acm.org/doi/10.1145/2461121.2461133

RESEARCH-ARTICLE

Understanding users in the wild

AITOR APAOLAZA, The University of Manchester, Manchester, Greater Manchester, U.K.

SIMON HARPER, The University of Manchester, Manchester, Greater Manchester, U.K.

CAROLINE JAY, The University of Manchester, Manchester, Greater Manchester, U.K.

Open Access Support provided by:

The University of Manchester

PDF Download
2461121.2461133.pdf
25 March 2026
Total Citations: 24
Total Downloads: 289

Published: 13 May 2013

Citation in BibTeX format

W4A '13: International Cross-Disciplinary Conference on Web Accessibility
May 13 - 15, 2013
Rio de Janeiro, Brazil

Understanding Users in the Wild

Aitor Apaolaza, Simon Harper and Caroline Jay
School of Computer Science
University of Manchester
Manchester, United Kingdom
[apaolaza|simon.harper|caroline.jay]@cs.manchester.ac.uk

ABSTRACT

Laboratory studies are a well established practice that present disadvantages in terms of data collection. One of these disadvantages is that laboratories are controlled environments that do not account for unpredicted factors from the real world. Laboratory studies are also obtrusive and therefore possibly biased. The Human-Computer Interaction (HCI) community has acknowledged these problems and has started exploring in-situ observation techniques. These observation techniques allow for bigger participant pools and their environments can conform to the real world. Such real-world observations are particularly important to the accessibility community who has coined the concept accessibility-in-use to differentiate real world from laboratory studies. Real-world observations provide low-level interaction data therefore making a bottom-up analysis possible. This way behaviours emerge from the obtained data instead of looking for predefined models. Some in-situ techniques employ Web logs in which the data is too coarse to infer meaningful user interaction. In some other cases an exhaustive manual modification is required to capture interaction data from a Web application. We describe a tool which is easily deployable in any Web application and captures longitudinal interaction data unobtrusively. It enables the observation of accessibility-in-use and guides the detection of emerging tasks.

Categories and Subject Descriptors

H.5.2 [**User Interfaces**]: Evaluation/methodology, Ergonomics; H.5.4 [**Hypertext/Hypermedia**]: User issues

General Terms

Human factors

Keywords

Ergonomics, Accessibility, Web, Usage mining, User issues, User behaviour analysis

Permission to make digital or hard copies of all or part of this work for personal or classroom use is granted without fee provided that copies are not made or distributed for profit or commercial advantage and that copies bear this notice and the full citation on the first page. To copy otherwise, to republish, to post on servers or to redistribute to lists, requires prior specific permission and/or a fee.
W4A2013 - Communication, May 13-15, 2013, Rio de Janeiro, Brazil.
Co-Located with the 22nd International World Wide Web Conference.
Copyright 2013 ACM 978-1-4503-1844-0 ...$15.00.

1. INTRODUCTION

Laboratory studies, commonly employed in interface evaluations, have a number of disadvantages. They employ controlled environments where all confounding factors are controlled. Users' day-to-day environments differ greatly from the ones found in laboratories. Everything from their computer set-up to their routine are unpredicted factors that are disregarded in these studies. The employed observing agent is a problem as well. The users are aware they are being observed, therefore making them behave in a different way. Effects like guinea pig effect or role model selection can bias the collected information. The HCI community started exploring other kind of observations that would overcome this disadvantages, like remotely ran in-situ observations. These observations take place in users' own environments and their remote aspect allows them to get to more participants than the ones able to physically participate in a laboratory.

The importance of these kind of real-world observations was acknowledged by the accessibility community. The concept of accessibility-in-use describes observations of real users in the real world trying to achieve real goals [13]. Accessibility-in-use observations are in-situ and unobtrusive, resulting in ecological observations of users' real interaction with Web applications. In order to analyse the real use of an interface top-down approaches with predetermined tasks models need to be avoided. Using predetermined tasks implies a prejudgement of the users' interaction. Collecting all low-level interaction data from users will enable the detection of emerging tasks. These tasks will represent what the user is really doing, as opposed to the tasks the designers planned the interface for.

As far as we know, current techniques cannot provide insight into accessibility-in-use. Some techniques use Web logs, giving an unobtrusive view of users. Although these Web logs can give interesting information like users' click-through, they make it impossible to identify any meaningful low interaction data from users. Other approaches requiring the installation of software result in obtrusive observations, in which the users need to be an active part in the observation. An alternative to get interaction data unobtrusively is thoroughly modifying the Web application so identified events are recorded. Selected events to be recorded can be semantically relevant for the specific Web application instead of generic interaction events, simplifying the analysis. The disadvantage of this approach is being cumbersome and not easily scalable.

The solution described in this paper allows an easy deployment into any Web application. It just requires adding

a JavaScript to all the Web pages of the site resulting in a very scalable solution. It captures low-level interaction data longitudinally in an unobtrusive way. This way it gives insight into accessibility in use, providing ecological data from the real world. Users' tasks emerge from the collected data, and their combination will result in ecologically valid behaviour models. Analysing behaviour models will raise accessibility issues which could then be fixed with the help of existing guidelines. Demographically similar users or users with similar disabilities may encounter the same problems. We find there will be no need to collect specific information about users – such as specific disabilities – as their accessibility problems will be found in the emerging behaviours.

2. RELATED LITERATURE

A considerable amount of literature has been published on the topic of understanding users. Our work focuses in observing users' interaction, to then shed light on the use of the Web site by analysing the obtained data. The interaction data is obtained from an agent logging what the users are doing. They can vary in diverse degrees of obtrusiveness, from a researcher taking notes while the user describes what he is doing in the think aloud protocol, to the use of Web server logs, not interfering with the user. The use of proxies to get the same information as Web server logs has been exploited in terms of showing paths and measuring timings at each Web page [5]. There have been visualization approaches that gave easily understandable insight into the users' behaviour performing a series of task [4], as well as methods that avoid predefining the users' interaction by letting them interact freely to then isolate the tasks to be evaluated from the data [12]. As we stated before, both approaches are biased as they only give insight into the use of those predefined task models. Other techniques combined event and video recording with surveys [9]. This enabled the drill-down from general statistics to video playback in order to detect usability problems in video games. Although shown to be valid this technique did not enable either longitudinal or unobtrusive observation.

Obtaining user models is another way to understand user behaviour. Developing user models from laboratory data may result in systems that can be used to predict real users' behaviour. Cogtool [8] is an example of a combination of different well established predictive metrics like Keystroke Level Model (KLM) [3] and Fitts' law. It provides a framework in which HTML pages can be designed, and several metrics regarding time aspects are provided.

Ample research has investigated the use of SERPs (Search Engine Result Pages) in order to understand users' motivations in a remote manner. These findings are not generalisable, and try to find the motivation of the users' behaviour rather than accessibility problems. Low-level interaction data captured via a modified toolbar can help finding out under what situations the user is going to click in an advertisement [6]. This knowledge helps in the decision of who to show the advertisement to and when. A bigger picture, even if lacks detail about interaction, can provide information about social trends. Massive registries of search queries over long periods of time can be employed to predict changes over time, like periodicities and popularity spikes [10].

Research into general use of the Web can vary in different degrees of detail as well. On the one hand, high-level longitudinal studies of particular users can give a glance into users' behaviour to then group them according to their use. Insight into the way these groups use the Web can be used to design high-level improvement strategies oriented at each group [11]. On the other hand, the discovered behaviour may be concise and specific to a particular situation that can be found in different type of Web sites. For example, finding out what characteristics of a displayed banner make it more appealing to users in order to catch their eye gaze [7].

3. IDENTIFIED REQUIREMENTS

As far as we know, current techniques do not provide the means to obtain real-world accessibility information of a Web application. Previous studies do not measure how well interfaces adapt to the users, but the other way around. Most of the previous work is biased by prejudgements of user interaction. Top down approaches use predefined task models that limit the evaluation to the assumed users' behaviour. Surveys can be used after allowing the user free interaction, but they are obtrusive and tedious for the user, making a longitudinal approach infeasible. User modelling does not suffer from these drawbacks, but the empirical data used to model its predictions usually comes from laboratory studies.

An accesibility-in-use observation technique is completely unobtrusive to avoid any biases. It provides in-situ longitudinal observations of the users. Users' evolve over time, so without long term observation factors like learnability are lost. We acknowledge the disadvantages of such an observation technique. Control over the environment is lost. Ambiguous situations like guessing what is happening during users' inactivity remain unclear. The advantage of collecting low-level interaction data in these situations is that finer data is available – like frequent mouse data –, making this episodes more brief. Therefore the distinction between a reading action and being away from the computer becomes clearer. Another problem is privacy, as it collects data from users without them noticing it. The collected data is not analysed for personal information that could lead into privacy issues. User profiles are to be kept only to take into account users' evolution, avoiding maintaining single profiles across different Web applications. Finally, we do not consider accessibility-in-use observation a substitute for previous qualitative and quantitative approaches. On the contrary, the combination of the two can help detecting and resolving accessibility problems as it has been seen elsewhere [14].

4. DESIGN OF THE SOLUTION

In order to get insight into accessibility-in-use the appropriate approach for data collection is needed. The identified requirements demand the solution to be unobtrusive and capable of collecting low-level interaction data in a longitudinal way. We firstly carried out a formative experiment in which we compared two different approaches in a real setting. Below we detail both approaches and the basis of the comparison. After making the comparison we implemented the capture solution, which we describe in this section.

4.1 Formative study

A formative study has been carried out to compare two possible logging solutions. These solutions were deployed

into a Web application[1] used by real users and the collected interaction data was compared in order to see which logging approach would be used. The two techniques evaluated were (1) *annotation of the Web application* and (2) *injection of JavaScript*.

The *annotation of the Web application* entailed manually extending the code of an already running Web application. The modification listened to specific events and logged them to a text file for future analysis. This technique is not generalisable as it is hard-coded, but enables the customisation of the events to gather, avoiding information overhead.

The *injection of JavaScript* technique consisted in a modification of UsaProxy [1]. This tool is a proxy which sits between the user and the Web site and inserts JavaScript code in all the Web pages served to the user. This JavaScript code records all low-level events from the user, from mouse movements and key presses to HTTP requests. UsaProxy has been widely used, having been employed in accessibility research before [2].

UsaProxy was modified so it could be used without requiring the configuration of a proxy. This modification allowed us to very easily modify the source code of the Web pages to insert the JavaScript code directly. This helped tackling the privacy concerns of recording all user interactions with non registered Web applications, limiting the recording to the analysed Web page. It also ensured that the Web application would keep working even if the server receiving the data failed, as opposed to what would happen when using a proxy.

The aspects to compare between the two approaches were their scalability and the amount of interaction data collected. The annotation approach allows to customise the collected events, but it does not automatise the procedure. On the other hand, the way the injection approach collects data will require a much more complex analysis solution, but it supports greater insight into behaviour.

As a result of this study we found that the annotation gave from the start more high-level understanding of the use of the Web site. As this high-level understanding comes from the ability to customise which events to listen, we modified the injection approach so it would listen to the same events. The injected JavaScript code was extended so it would take into account the DOM paths of the elements that triggered the mentioned hard-coded events. This way the latter approach was able to give the same degree of high-level information without the need of modifying the source of the Web site.

Customised events can be identified using their DOM path and the event type that triggers them. In our modification, the identification was performed modifying the JavaScript code. This process can also take place after the collection of data, as it contains all the necessary data for identifying those events. Even if recording specific hard-coded events is not our objective, this experiment showed that the information captured by the injection method was richer. It gives both the ability of recording an immense amount of low-level events and the preciseness of being able to define Web site specific relevant events.

4.2 Implementation

Once the approach to take for the logging tool was decided, we modified the UsaProxy tool so it would fit the necessities of our solution. In order to be able to record as much interaction as possible from dynamic Web applications, new events were added. For example, mouse wheel was found useful to capture not only the overall window's scroll, but wheel movements in frames that may have their own scrollbar. Other events were modified, either to change their behaviour, like the way the key-presses are recorded, or to add new fields, like the client's browser name. We have tested the application across different browsers, but recording this information would help managing possible incompatibilities with new browser versions.

In order to be able to maintain users' profiles over long periods of time, cookies were necessary. We modified the way cookies were originally handled by UsaProxy, so the cookies were deployed via the JavaScript. This makes them only accessible from that domain, preventing the profiling of specific users across different Web sites. Maintaining users' profile across Web sites would allow us to classify them according to their browsing behaviour. We find this outside the scope of the solution, apart from creating unnecessary privacy problems. Profiling users across different Web sites can raise ethical issues. Even if the data is anonymised, the list of accessed Web sites over time may breach personal privacy. Another reason was keeping the cookie in the Web site's domain. This way it's the Web site's responsibility to obtain users' permission as another standard cookie if required.

The implemented solution is easily deployable in any Web site or Web application. The only required modification is adding a small script to every Web page of the site. The script then gets executed, constructs the JavaScript declaration, and adds the corresponding JavaScript declaration when the Web page is accessed.

Once the JavaScript is deployed, it starts sending events to the server. Even if the server receiving collected data is down, the Web application carries on working without producing any errors. Keyboard, mouse and window modification related events are registered and recorded every time they are triggered. Mouse movement events are recorded periodically every 150 ms. All mouse wheel events happening within 100 ms are grouped and the scroll values get added to avoid recording unnecessary events.

Once the server receives interaction data, it gets processed and stored in a NoSQL database. In Figure 1 two examples of captured data for key-down and mouse-move events are depicted. The data available differ between different kind of events. Information about the DOM node where the action took place, mouse coordinates, and keyboard information are recorded only if they are available for the particular event. For example, a mouse event will not contain any keyboard information, in the same way that the event of loading a new Web page is not related to a particular node.

Knowing the state of the Web application at any point of the interaction is necessary. This way a drill-down will be possible and a reconstruction of the users' interaction episode will be possible for detailed understanding. In Web logs the only way to detect a state change is looking at data requests. In some Web pages, interactive elements can change the state of the page without the need of any more data. For example, in a tabbed application all tabs may load at start-time for the user to switch between them seamlessly. That tab change would not trigger a new data request, but would provoke a change in the DOM. Our solution sends the

[1] www.kupkb.org

```
{ "_id" : ObjectId("511be744e4b02784a6646188"),
  "ip" : "130.88.193.26",
  "timestamp" : "2013-02-13,19:19:31:432",
  "sd" : "10001",
  "sid" : "b4VbZ1YProPQ",
  "event" : "keypress",
  "key" : "d",
  "nodeInfo" : {
    "nodeDom" : "id(%22eps-schools%22)",
    "nodeType" : "BODY",
    "nodeTextValue" : "undefined"
    },
  "browser" : "Chrome24.0.1312.70",
  "url" : "http%3A%2F%2Fwww.cs.man.ac.uk%2F
    ~apaolaza%2FTESTindex.html"
}

{ "_id" : ObjectId("511be744e4b02784a6646186"),
  "ip" : "130.88.193.26",
  "timestamp" : "2013-02-13,19:19:31:216",
  "sd" : "10001",
  "sid" : "b4VbZ1YProPQ",
  "event" : "mousemove",
  "mouseCoordinates" : {
    "coordX" : 709,
    "coordY" : 248,
    "offsetX" : 417,
    "offsetY" : 3
    },
  "nodeInfo" : {
    "nodeDom" : "id(%22content%22)/P[2]",
    "nodeType" : "P",
    "nodeTextValue" : "undefined"
    },
  "browser" : "Chrome24.0.1312.70",
  "url" : "http%3A%2F%2Fwww.cs.man.ac.uk%2F
    ~apaolaza%2FTESTindex.html"
}
```


Figure 1: Example of the output.

DOM state to the server every time a registered event gets activated, in the current version it gets captured every time the user clicks. In the server the DOM is stored if it is the first time. Otherwise it gets compared to the previous state and the differences between them are stored. Constructing the DOM state of the user at any point is then possible by applying all the changes to the base DOM.

5. CONTRIBUTIONS AND CONCLUSIONS

The solution described in this paper provides a way to get in-situ understanding from users. It collects low-level interaction data longitudinally in an unobtrusive way. Instead of using predetermined task models, users' own tasks from real use emerge from the data. These tasks will be combined resulting into models which will be analysed for accessibility issues. We acknowledge previous qualitative and quantitative approaches' value, and we think they can be complemented with the described approach. The advantages of using a longitudinal unobtrusive technique are threefold: evaluation in the wild is possible, thus reaching more users; the validity of the results will not get compromised due to the unobtrusive observation; and it will grant the ability to ecologically know what is happening as opposed to a laboratory environment.

6. REFERENCES

[1] R. Atterer, M. Wnuk, and A. Schmidt. Knowing the user's every move: user activity tracking for website usability evaluation and implicit interaction. In *Proceedings of the 15th international conference on World Wide Web*, WWW '06, pages 203–212, 2006.

[2] J. P. Bigham, A. C. Cavender, J. T. Brudvik, J. O. Wobbrock, and R. E. Lander. WebinSitu: a comparative analysis of blind and sighted browsing behavior. In *Proceedings of the 9th international ACM SIGACCESS conference on Computers and accessibility*, Assets '07, pages 51–58, 2007.

[3] S. K. Card, T. P. Moran, and A. Newell. The keystroke-level model for user performance time with interactive systems. *Commun. ACM*, 23(7):396–410, July 1980.

[4] T. Carta, F. Paternò, and V. Santana. Support for remote usability evaluation of web mobile applications. In *Proceedings of the 29th ACM international conference on Design of communication*, SIGDOC '11, pages 129–136, 2011.

[5] J. Cugini and J. Scholtz. VISVIP: 3D visualization of paths through web sites. In *Database and Expert Systems Applications, 1999. Proceedings. Tenth International Workshop on*, pages 259 –263, 1999.

[6] Q. Guo, E. Agichtein, C. L. A. Clarke, and A. Ashkan. In the mood to click? towards inferring receptiveness to search advertising. In *Proceedings of the 2009 IEEE/WIC/ACM International Joint Conference on Web Intelligence and Intelligent Agent Technology - Volume 01*, WI-IAT '09, pages 319–324, 2009.

[7] C. Jay, A. Brown, and S. Harper. Predicting whether users view dynamically updating content on the world wide web. *ACM Transactions on Computer-Human Interaction*, To appear, 2013.

[8] B. E. John, K. Prevas, D. D. Salvucci, and K. Koedinger. Predictive human performance modeling made easy. In *Proceedings of the SIGCHI conference on Human factors in computing systems*, CHI '04, pages 455–462, 2004.

[9] J. H. Kim, D. V. Gunn, E. Schuh, B. Phillips, R. J. Pagulayan, and D. Wixon. Tracking real-time user experience (TRUE): a comprehensive instrumentation solution for complex systems. In *Proceedings of the twenty-sixth annual SIGCHI conference on Human factors in computing systems*, CHI '08, pages 443–452, 2008.

[10] K. Radinsky, K. Svore, S. Dumais, J. Teevan, A. Bocharov, and E. Horvitz. Modeling and predicting behavioral dynamics on the web. In *Proceedings of the 21st international conference on World Wide Web*, WWW '12, pages 599–608, 2012.

[11] C. Tossell, P. Kortum, A. Rahmati, C. Shepard, and L. Zhong. Characterizing web use on smartphones. In *Proceedings of the 2012 ACM annual conference on Human Factors in Computing Systems*, CHI '12, pages 2769–2778, 2012.

[12] A. Vargas, H. Weffers, and H. V. da Rocha. A method for remote and semi-automatic usability evaluation of web-based applications through users behavior analysis. In *Proceedings of the 7th International Conference on Methods and Techniques in Behavioral Research*, MB '10, pages 19:1–19:5, 2010.

[13] M. Vigo and G. Brajnik. Automatic web accessibility metrics: Where we are and where we can go. *Interacting with Computers*, 23(2):137 – 155, 2011.

[14] M. Vigo and S. Harper. Evaluating accessibility-in-use. In *Proceedings of the 2013 International Cross Disciplinary Conference on Web Accessibility (W4A)*, W4A '13, 2013.